Práctica 1: Aprendizaje por Refuerzo

Gloria Casals Zapardiel
gloria.casals@um.es

Ángel Franco Rodríguez
angel.f@um.es

9 de marzo de 2026

Resumen

En este trabajo se aborda una progresión en el Aprendizaje por Refuerzo, comenzando por la toma de decisiones estáticas bajo incertidumbre hasta el control en entornos dinámicos y continuos. Inicialmente, estudiamos el problema del bandido de k-brazos para analizar el dilema exploración-explotación. Evaluamos métodos de acción-valor (ϵ-greedy), límite superior de confianza (UCB) y gradiente (Softmax) bajo distintas distribuciones de recompensa (Normal, Binomial y Bernoulli). Los resultados de esta fase demuestran que la eficacia algorítmica depende directamente de la varianza y naturaleza de la señal de recompensa.

Posteriormente, extendemos el estudio a la toma de decisiones secuenciales mediante Procesos de Decisión de Markov (MDP) en espacios discretos (*Blackjack* y *CliffWalking*). La implementación de métodos tabulares de Monte Carlo y Diferencias Temporales (SARSA y Q-Learning) evidencia cómo la temporalidad de los incentivos (episódica frente a penalizaciones paso a paso) y el enfoque algorítmico (*On-Policy* frente a *Off-Policy*) afectan la convergencia y el nivel de conservadurismo de la política aprendida.

Finalmente, abordamos el aumento de la dimensionalidad en espacios de estados continuos (*MountainCar*) mediante la aproximación de funciones con redes neuronales.

Índice general

1. Problema del Bandido de k-brazos **3**

1.1. Introducción . . . 3

1.2. Desarrollo . . . 3

1.3. Algoritmos . . . 3

1.3.1. Algoritmo ϵ-greedy . . . 4

1.3.2. Algoritmo ϵ-decay (Decaimiento) . . . 4

1.3.3. Algoritmo UCB1 (Upper Confidence Bound) . . . 4

1.3.4. Algoritmo UCB2 . . . 4

1.3.5. Algoritmo Softmax . . . 4

1.4. Evaluación/Experimentos . . . 4

1.4.1. Configuración experimental y métricas . . . 4

1.4.2. Sensibilidad del parámetro ε en algoritmos ε-greedy . . . 5

1.4.3. Análisis comparativo entre algoritmos ϵ-greedy, ϵ-decaimiento, UCB1, UCB2 y Softmax . . . 5

1.5. Conclusiones . . . 6

2. Aprendizaje en Entornos Complejos: Técnicas tabulares y aproximadas de Aprendizaje por Refuerzo **7**

2.1. Introducción . . . 7

2.2. Desarrollo . . . 7

2.2.1. Métodos tabulares . . . 8

2.2.2. Métodos aproximados . . . 8

2.3. Algoritmos . . . 8

2.3.1. Métodos Tabulares . . . 8

2.3.2. Aproximación de Funciones (Deep RL) . . . 9

2.4. Evaluación/Experimentos . . . 9

2.4.1. Configuración Experimental . . . 9

2.4.2. Resultados en entornos tabulares . . . 9

2.4.3. Resultados en MountainCar (aproximación de funciones) . . . 11

2.5. Conclusiones 11

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

La viabilidad económica de líderes de la industria como Alphabet y Meta reside en la resolución eficiente de un problema de optimización constante: la personalización de anuncios. El reto consiste en identificar qué creatividad publicitaria maximiza la probabilidad de interacción para cada usuario. Lograrlo no solo incrementa los ingresos, sino que provoca que la plataforma sea percibida como un servicio crítico para las empresas que se anuncian. A lo largo de esta práctica vamos a modelar esta situación utilizando el marco teórico del Bandido Multibrazo, simulando un entorno donde contamos con k alternativas y buscamos maximizar la recompensa total obtenida.

Este escenario representa la encarnación perfecta del dilema del aprendizaje por refuerzo: el conflicto entre la **exploración** (probar nuevos anuncios para evaluar su eficacia) y la **explotación** (mostrar el anuncio con mejor rendimiento conocido hasta el momento) .

El objetivo de este informe es implementar y evaluar empíricamente distintas familias de algoritmos propuestos en la literatura para resolver este problema. Mediremos la capacidad de cada agente para minimizar el arrepentimiento acumulado (regret) y maximizar la selección de la acción óptima bajo tres distribuciones con recompensas distintas: Normal, Binomial y Bernoulli.

1.2. Desarrollo

El problema del **bandido de k-brazos** modela la toma de decisiones secuenciales bajo incertidumbre. Formalmente, se define por la tupla $\langle A, \mathcal{R} \rangle$ caracterizada por los siguientes elementos:

- Un conjunto de k acciones o "brazos": $A = \{a_1, a_2, \ldots, a_k\}$.
- Un horizonte temporal discreto indexado por $t = 1, 2, \ldots, T$. En cada paso t:
  - El agente selecciona una acción $A_t \in A$.
  - El entorno devuelve una recompensa R_t extraída de una distribución de probabilidad desconocida asociada a dicha acción: $R_t \sim P(r|A_t)$.
- El objetivo es encontrar una política de selección que maximice la recompensa acumulada esperada:

$$\text{Maximizar } \mathbb{E}\left[\sum_{t=1}^{T} R_t\right] \tag{1.1}$$

Esta formulación fue introducida formalmente por [5] y representa el conflicto fundamental entre **exploración** (recopilar datos sobre brazos desconocidos) y **explotación** (seleccionar el brazo que se cree óptimo).

Para su resolución, la literatura clasifica las soluciones en las siguientes familias:

- **Métodos de Acción-Valor:** estiman el valor empírico de cada acción. La estrategia ϵ-greedy explota la mejor estimación actual, pero introduce una probabilidad ϵ para explorar aleatoriamente, presentando la desventaja de una exploración ciega.
- **Métodos de Índice (UCB):** incorporan una clase genérica de políticas de índice denominadas límites superiores de confianza como medida de incertidumbre, de forma que se escoge el brazo que maximiza la combinación entre la recompensa esperada y un término de confianza. De este caso existen variantes: UCB1, UCB2.
- **Métodos de Gradiente (Softmax):** asignan preferencias numéricas a las acciones, transformándolas en probabilidades de selección mediante una función de Gibbs. Los brazos con mejor rendimiento tienen mayor probabilidad de elección, manteniendo una exploración sensible al valor relativo.
- **Enfoques Bayesianos:** modelan la incertidumbre de los parámetros de recompensa con distribuciones de probabilidad (a priori y a posteriori), eligiendo acciones de forma proporcional a su probabilidad de ser óptimas (por ejemplo Muestreo de Thompson).
- También existen variaciones del problema como los bandidos contextuales (las recompensas dependen de información del entorno) o bandidos no estacionarios (las recompanesas cambian con el tiempo) que adecúan el problema a situaciones reales y que también tienen diferentes enfoques.

En este trabajo evaluaremos el comportamiento de los algoritmos ϵ-greedy, UCB y Softmax. El desempeño se medirá utilizando tres métricas: la recompensa promedio, el porcentaje de selección de la acción óptima y la evolución del arrepentimiento (regret) acumulado.

1.3. Algoritmos

La elección de este conjunto de algoritmos permite tener una progresión en la gestión de la incertidumbre y nos da un mayor campo de experimentación donde entender dicha incertidumbre y qué relación guarda con según qué distribución seguida

por los brazos.

1.3.1. Algoritmo ϵ-greedy

Este método aborda el dilema de exploración-explotación mediante una política casi determinista. Se escoge el brazo con mayor recompensa estimada, dejando una probabilidad constante ϵ para la exploración aleatoria. Formalmente:

$$A_t = \begin{cases} \arg\max_a Q_t(a), & \text{con probabilidad } 1-\varepsilon \\ \text{acción aleatoria}, & \text{con probabilidad } \varepsilon \end{cases}$$

La desventaja de este enfoque es su ineficiencia asintótica: continúa explorando ciega e indefinidamente brazos subóptimos incluso cuando ya posee suficiente información empírica.

1.3.2. Algoritmo ϵ-decay (Decaimiento)

Esta variante optimiza el método anterior forzando a que el parámetro ϵ disminuya con el tiempo. Se comienza con una alta exploración inicial que decae hacia la explotación pura conforme se consolida el conocimiento del entorno. El decaimiento puede ser lineal, exponencial o inversamente proporcional:

$$\varepsilon_t = \frac{\varepsilon_0}{1+\lambda t} \quad \text{ó} \quad \varepsilon_t = \varepsilon_0 \cdot e^{-\lambda t}$$

1.3.3. Algoritmo UCB1 (Upper Confidence Bound)

A diferencia de ϵ-greedy, que explora al azar, UCB1 explora con optimismo ante la incertidumbre. La fórmula de selección es determinista, y selecciona el brazo que maximiza un límite superior de confianza:

$$A_t = \operatorname*{argmax}_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

- El primer término ($Q_t(a)$) fomenta la explotación (elige el mejor).
- El segundo término (la raiz cuadrada) fomenta la exploración de los brazos menos visitados. Normalmente, y como hemos visto en clase, se coge $c = \sqrt{2}$ para obtener cotas de Hoeffding.

Si un brazo no se ha visitado nunca ($N_t(a) = 0$) el término de la raíz tiende a infinito, por lo que el algoritmo debe seleccionarlo antes de aplicar la fórmula.

1.3.4. Algoritmo UCB2

El algoritmo UCB2 es una extensión de UCB1 que pretende mejorar la frecuencia de exploración mediante fases de selección (epochs). En lugar de recalcular la acción óptima en cada paso, este algorimto selecciona un brazo y lo juega varias veces antes de tomar una elección diferente.

$$A_t = \arg\max_a \left[Q_t(a) + \sqrt{\frac{(1+\alpha)\,\ln\left(\frac{e\,t}{\tau(k_a)}\right)}{2\,\tau(k_a)}} \right]$$

Donde $\alpha \in (0,1)$ controla la exploración y $\tau(k_a) = \lceil (1+\alpha)^{k_a} \rceil$ determina la longitud de la época para el brazo a, siendo k_a su contador de épocas.

1.3.5. Algoritmo Softmax

Abandona la dicotomía entre elegir el mejor brazo o elegir al azar. La probabilidad de elegir un brazo es proporcional a su valor estimado:

$$P(a) = \frac{e^{Q_t(a)/\tau}}{\sum_{i=1}^{k} e^{Q_t(i)/\tau}}$$

El hiperparámetro τ (temperatura) regula la discriminación:

- Si $\tau \to \infty$, $P(a)$ tiende a una distribución uniforme (exploración pura).
- Si $\tau \to 0$, la selección tiende a ser *greedy* (explotación pura).

$$\pi_t(a) = \frac{e^{Q_t(a)/\tau}}{\sum_{i=1}^{k} e^{Q_t(i)/\tau}}$$

Justificación del parámetro τ

Para los entornos simulados con recompensas normalizadas o Bernoulli, hemos seleccionado $\tau = 0{,}1$. Esto da una alta sensibilidad discriminativa. Matemáticamente, ante una diferencia pequeña en las estimaciones (por ejemplo $\Delta Q = 0{,}1$) entre el brazo óptimo a^* y uno subóptimo a_i, el ratio de probabilidades es:

$$\frac{P(a^*)}{P(a_i)} = e^{\frac{\Delta Q}{\tau}} = e^{\frac{0{,}1}{0{,}1}} \approx 2{,}71$$

Esto implica que el brazo óptimo es casi tres veces más probable de ser elegido, favoreciendo una convergencia rápida sin truncar prematuramente la probabilidad de rectificación.

1.4. Evaluación/Experimentos

1.4.1. Configuración experimental y métricas

La configuración experimental está formada por los siguientes parámetros:

- **Entorno:** 10 brazos independientes. Para garantizar la robustez del análisis se simulan tres escenarios con diferentes distribuciones de recompensa: Normal ($\mu \in [0,10], \sigma = 3$), Binomial y Bernoulli.
- **Ejecución:** Los resultados se promedian sobre 500 ejecuciones independientes de $T = 1000$ pasos de tiempo cada una.
- **Métricas:** Se evalúan: (1) Recompensa promedio, (2) Porcentaje de selección de la acción óptima, y (3) Evolución del arrepentimiento (regret) acumulado.

1.4.2. Sensibilidad del parámetro ε en algoritmos ε-greedy

En una primera fase hemos contrastado el rendimiento del algoritmo ϵ-greedy para $\epsilon = 0$, $\epsilon = 0{,}01$, $\epsilon = 0{,}1$.

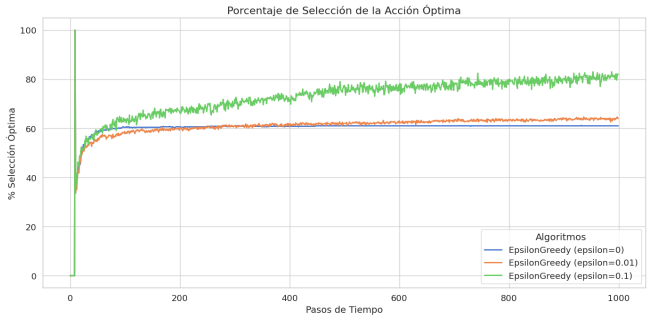

Figura 1.1: Porcentaje de selección óptima para los 3 valores de ε bajo una Distribución Normal.

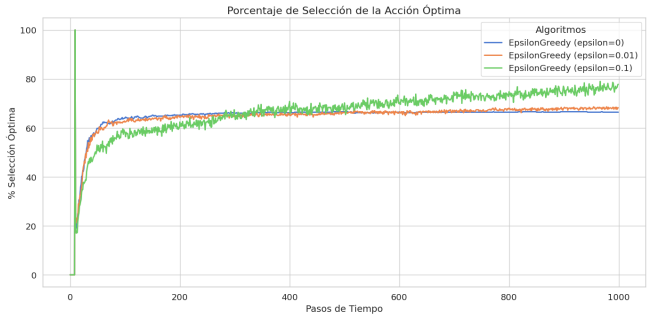

Figura 1.2: Porcentaje de selección óptima para los 3 valores de ε bajo una Distribución Bernoulli.

Los resultados nos muestran que el tipo de distribución determina la velocidad de aprendizaje. Las recompensas continuas de la distribución Normal exigen mayor exploración para aislar el brazo óptimo frente a la variabilidad, mientras que la distribución Binomial ofrece un aprendizaje más estable y progresivo. Por úlitmo, la distribución de Bernoulli es la que más tarda en encontrar la acción óptima, con una convergencia más ruidosa y volátil, ya que sus recompensas son binarias y la información obtenida por tanto es limitada.

A pesar de estas diferencias de velocidad, el comportamiento de ε es consistente en los tres escenarios:

- $\boldsymbol{\varepsilon = 0}$: obtiene una recompensa promedio alta rápidamente, aunque puede no alcanzar el óptimo por falta de exploración.
- $\boldsymbol{\varepsilon = 0{,}1}$: realiza más exploración, por lo que dándole tiempo llegaría a la acción óptima, aunque el regret acumulado es alto debido a la aleatoriedad
- $\boldsymbol{\varepsilon = 0{,}01}$: logra un equilibrio entre los dos enfoques anteriores, maximizando el hallazgo de la mejor acción con una penalización mínima en el *regret* a largo plazo.

1.4.3. Análisis comparativo entre algoritmos ϵ-greedy, ϵ-decaimiento, UCB1, UCB2 y Softmax

Los resultados nos muestran que la distribución de la recompensa determina la eficacia:

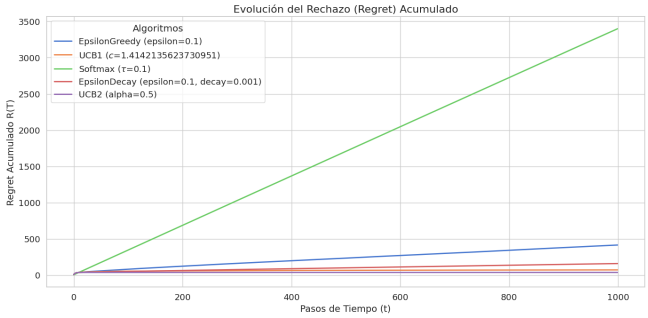

Figura 1.3: Evolución del regret acumulado para cada algoritmo bajo una Distribución Normal.

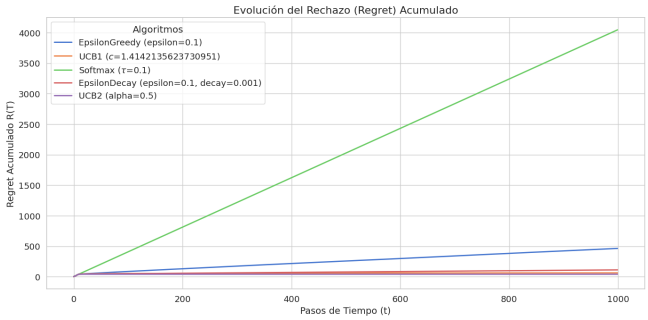

Figura 1.4: Evolución del regret acumulado para cada algoritmo bajo una Distribución Binomial.

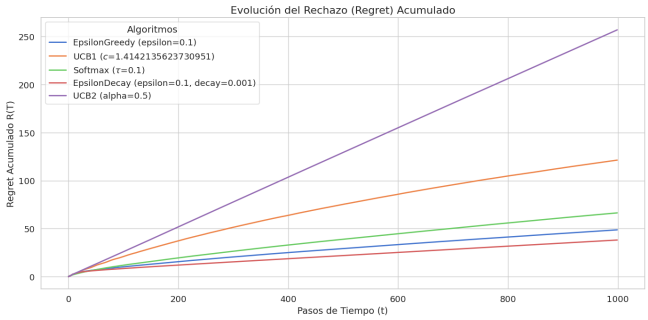

Figura 1.5: Evolución del regret acumulado para cada algoritmo bajo una Distribución Bernoulli.

1. Entorno de recompensas Normales (Continuas): En escenarios de alta varianza de la recompensa, los métodos de índice destacan positivamente. UCB2 se posiciona como el mejor algoritmo, seguido de UCB1, gracias a su capacidad para ajustar la exploración basándose en la incertidumbre. Podemos ver en la Figura 1.3 como UCB2 y UCB1 logran aplanar su curva rápidamente, demostrando un equilibrio óptimo entre exploración e incertidumbre. Los métodos voraces (ϵ-decay y ϵ-greedy) logran resultados competitivos, aunque son más dependientes de una buena exploración inicial. Finalmente, Softmax es más gradual, ya que explora las acciones con probabilidades proporcionales a su recompensa estimada, provocanddo una baja velocidad de convergencia, aunque mayor estabilidad.

2. Entorno de recompensas Binomiales (Discretas): La varianza moderada de esta distribución estabiliza el com-

portamiento de todos los algoritmos. La jerarquía se mantiene: UCB2 y UCB1 siguen siendo los mejores al reducir la exploración inútil. Sin embargo, ϵ-decay reduce la brecha de rendimiento frente a los métodos UCB, y Softmax vuelve a mostrar el peor desempeño, superando incluso el rechazo de la distribución Normal (alcanzando más de 4000 de regret). Esto confirma que, a escalas de recompensa similares, la discretización de la Binomial no altera la dinámica de exploración-explotación de estos algoritmos.

3. Entorno de recompensas Bernoulli (Binarias): La escasez de información de la recompensa binaria invierte la clasificación de eficiencia. En este contexto, ϵ-decay es el mejor algoritmo. Su transición rápida de una fase exploratoria a una de explotación resulta ser la estrategia óptima para mitigar el ruido. En contraparte, los UCB pasan a un segundo plano pues la falta de información de esta distribución se traduce en alta incertidumbre y por ello incluso en unos primeros instantes vemos algún pico de recompensa pero los algoritmos UCB deciden seguir explorando, retrasando su éxito. Por último, Softmax pasa a ser una solución intermedia, útil cuando las diferencias entre brazos no son muy grandes.

1.5. Conclusiones

Resumen de los principales resultados

El análisis demuestra que no existe un algoritmo universalmente superior; la eficacia de cada método depende de la naturaleza de la recompensa. En entornos con recompensas continuas o de varianza moderada (Normal y Binomial), los métodos basados en el optimismo ante la incertidumbre (UCB1 y UCB2) dominan al gestionar eficientemente la exploración. Sin embargo, en escenarios de baja información con recompensas binarias (Bernoulli), la alta varianza perjudica a los métodos UCB provocando una sobreexploración perjudicial. En este contexto, las transiciones rápidas de exploración a explotación, como las proporcionadas por ϵ-decay, resultan ser la estrategia óptima para reducir el ruido y minimizar el arrepentimiento acumulado.

Reflexión sobre la importancia y el impacto

Este trabajo creemos que muestra perfectamente por qué el problema del bandido multibrazo es muy importante en el Aprendizaje por Refuerzo, ya que podemos evidenciar como una señal binaria (como un clic en un anuncio de Meta o Alphabet) requiere un tratamiento algorítmico distinto al de una señal continua (como el tiempo de visualización en un vídeo de YouTube). En este trabajo hemos visto la importancia de alinear el diseño del agente con las propiedades del entorno para obtener un mejor retorno.

Limitaciones del estudio y posibles mejoras futuras

La principal limitación de este estudio es asumir un entorno *estacionario*, donde las distribuciones de recompensa de los brazos permanecen inalteradas en el tiempo. También una mejora inmediata consistiría en implementar una fase previa de optimización de hiperparámetros para cada tipo de distribución antes de realizar la comparativa del regret.

Líneas futuras de estudio

Para acercar el trabajo a aplicaciones reales más complejas, se proponen las siguientes vías de investigación:

- **Entornos no estacionarios:** modificar las reglas de actualización de los algoritmos para tener recompensas que evolucionan con el tiempo, por ejemplo modelando la fatiga del usuario ante un anuncio publicitario.
- **Bandidos contextuales:** extender el problema al caso donde el agente recibe información (el perfil del usuario) antes de tomar una decisión.

Capítulo 2

Aprendizaje en Entornos Complejos: Técnicas tabulares y aproximadas de Aprendizaje por Refuerzo

La longitud del episodio es un indicador de aprendizaje que complementa a la recompensa acumulada, pero su interpretación depende de la naturaleza del entorno:

- **En entornos de navegación y búsqueda de meta con penalización constante (como MountainCar y CliffWalking):** El objetivo principal es llegar a la meta en el menor tiempo posible, ya que cada paso extra conlleva un castigo. Al inicio del entrenamiento (alta exploración), el agente da muchos pasos deambulando al azar, cae repetidas veces por el acantilado o agota el tiempo máximo permitido (como el límite de 200 pasos en MountainCar). Conforme el agente aprende y la política converge hacia la óptima, la longitud del episodio disminuye y se estabiliza, indicando que el agente ha encontrado la ruta más directa, segura y eficiente hacia el objetivo.

- **En entornos episódicos de toma de decisiones estáticas (como Blackjack):** La dinámica cambia por completo respecto a la navegación espacial. En este caso la longitud del episodio (número de cartas pedidas o turnos) es muy corta por naturaleza y no es un indicador fiable de aprendizaje. Una política óptima puede dictar que el episodio dure un solo paso (plantarse inmediatamente con una buena mano) o varios (pedir cartas para acercarse a 21). Por lo tanto, el éxito del aprendizaje no se evalúa minimizando o maximizando la duración de la partida, sino observando la estabilización del retorno medio y el aumento en la tasa de victorias frente al crupier.

2.1. Introducción

Tras el estudio de los problemas de decisión estática con el problema de k-brazos, esta segunda parte de la práctica se centra en el Aprendizaje por Refuerzo (RL) aplicado a entornos dinámicos donde las decisiones tienen efectos a largo plazo. El RL permite a un agente aprender una política óptima a través de la interacción directa con un entorno, enfrentándose al desafío de maximizar una recompensa acumulada en escenarios donde el modelo de transición es desconocido.

El objetivo central de este bloque es doble. En primer lugar, profundizar en las técnicas tabulares, herramientas fundamentales para resolver Procesos de Decisión de Markov (MDP) cuando el espacio de estados y acciones es lo suficientemente reducido para ser representado explícitamente en memoria. En segundo lugar, superar las limitaciones analíticas de los métodos tabulares al enfrentarnos a entornos de alta dimensionalidad o con espacios de estados continuos, requiriendo el uso de Aproximación de Funciones mediante redes neuronales.

Para abordar esta progresión, utilizaremos entornos de la librería Gymnasium como banco de pruebas (tales como Blackjack, CliffWalking y MountainCar-v0). El trabajo se estructura en torno a la implementación, evaluación y análisis de distintos paradigmas de aprendizaje:

- **Métodos de Monte Carlo:** Donde el aprendizaje se basa en el promedio de los retornos empíricos obtenidos al finalizar cada episodio.
- **Diferencias Temporales (TD) Tabulares:** Métodos que combinan ideas de Monte Carlo y programación dinámica para actualizar las estimaciones de valor de forma incremental mediante bootstrapping.
- **Métodos Aproximados:** Adaptación de TD al caso continuo mediante la parametrización de funciones de valor, contrastando arquitecturas *On-Policy* (SARSA Semi-gradiente) frente a *Off-Policy* (DQN).

Analizaremos enfoques On-Policy (aprender de la política actual) y Off-Policy (aprender una política objetivo mientras se sigue otra de comportamiento), así como el impacto de la configuración de recompensas y la exploración en la convergencia del agente.

Con este enfoque no solo se busca la resolución técnica del problema, sino comprender la naturaleza de la Iteración de la Política Generalizada y cómo la definición de la función de recompensa afecta el comportamiento final del agente inteligente.

2.2. Desarrollo

Continuamos formalizando el marco de trabajo para las técnicas de aprendizaje por refuerzo. A diferencia de los problemas estáticos de n-brazos, el agente interactúa secuencialmente

con el entorno, de modo que cada acción influye tanto en la recompensa inmediata como en la evolución futura del estado. Nos situamos, por tanto, en el marco de los Procesos de Decisión de Markov (MDP), donde el objetivo general del agente es encontrar una política π que maximice el retorno esperado G_t.

Para abordar este desafío, el trabajo se divide en dos grandes bloques metodológicos condicionados por la naturaleza del espacio de estados $\mathcal{S}$ del entorno: métodos tabulares y métodos aproximados.

2.2.1. Métodos tabulares

Cuando el espacio de estados y acciones es finito y lo suficientemente reducido, es posible almacenar explícitamente las estimaciones de valor acción-estado $Q(s, a)$ en una tabla. Para evaluar este paradigma, hemos considerado dos entornos clásicos:

- **Blackjack-v1:** El estado viene determinado por una terna discreta (suma del jugador, carta visible del crupier, as usable), con dinámica estocástica.
- **CliffWalking:** El estado corresponde a la posición discreta en una cuadrícula, con una estructura de recompensas que penaliza severamente las caídas.

El desarrollo teórico se fundamenta en el esquema de Iteración de la Política Generalizada (GPI), que articula dos procesos acoplados: la evaluación de la política mediante la estimación de $Q(s, a)$, y la mejora de la política haciéndola progresivamente más voraz respecto a dicha estimación.

Para implementar este ciclo en un marco estrictamente tabular se consideran dos estrategias principales:

- **Monte Carlo (MC)**: estima el retorno completo G_t y actualiza los valores únicamente al finalizar cada episodio. Se trata de un estimador insesgado, aunque con alta varianza.
- **Diferencias Temporales (TD)**: actualiza las estimaciones tras cada transición observada, utilizando bootstrapping mediante los valores actuales. Introduce cierto sesgo, pero reduce la varianza y permite aprendizaje online.

2.2.2. Métodos aproximados

El enfoque tabular resulta inmanejable ante la alta dimensionalidad. Para evaluar entornos de alta complejidad introducimos el uso de redes neuronales como aproximadores de la función de valor $\hat{q}(S, A, \mathbf{w}) \approx q_\pi(S, A)$.

Hemos seleccionado el entorno **MountainCar-v0**. Este entorno presenta un espacio de estados continuo (posición $[-1{,}2, 0{,}6]$ y velocidad $[-0{,}07, 0{,}07]$) y un desafío adicional de recompensa dispersa (*sparse reward*), donde el agente recibe -1 por cada paso hasta alcanzar la meta, exigiendo una exploración eficiente.

La elección de la versión con acciones discretas (`MountainCar-v0`) en lugar de su variante continua (`MountainCarContinuous-v0`) se debe a que los algoritmos evaluados son de tipo value-based, por lo que requieren calcular la acción que maximiza el valor esperado, es decir, $\max_a Q(s, a)$. En un espacio de acciones continuo, esta optimización global no es posible sin recurrir a arquitecturas de tipo Actor-Crítico para control continuo, lo cual queda fuera del alcance de este trabajo.

Para resolver este MDP continuo, se han implementado y contrastado dos algoritmos:

- **SARSA Semi-gradiente (On-Policy):** Extiende TD al caso continuo. Actualiza los pesos de la red aprendiendo de la política en ejecución. Emplea el semi-gradiente, congelando los pesos en el cálculo del objetivo temporal para mitigar inestabilidades matemáticas.
- **DQN (Off-Policy):** Es una implementación de Deep Q-Learning de tipo Off-Policy. Busca converger a la política óptima independientemente de la política de exploración. Para evitar la divergencia de los pesos al combinar aproximación de funciones, *bootstrapping* y entrenamiento *off-policy*, este agente incorpora dos mecanismos:
  - *Experience Replay*: Un búfer de memoria que permite entrenar con mini-batches aleatorios, rompiendo la fuerte correlación temporal de las secuencias del MDP.
  - *Target Network*: Una red objetivo que se congela periódicamente para estabilizar el cálculo del error temporal.

2.3. Algoritmos

2.3.1. Métodos Tabulares

En esta primera aproximación, al tratar con espacios discretos y finitos, la función de valor $Q(s, a)$ se almacena de forma explícita

Monte Carlo

El aprendizaje por Monte Carlo (MC) estima los valores acción-estado promediando retornos completos observados al final de cada episodio. Para cada par (s, a), el valor se aproxima como:

$$Q(s, a) \approx \frac{1}{n} \sum_{i=1}^{n} G_i(s, a)$$

Las actualizaciones se realizan únicamente tras finalizar el episodio, lo que implica que el método no utiliza bootstrapping. En consecuencia, MC es un estimador insesgado, aunque puede presentar alta varianza.

El algoritmo sigue el esquema de Iteración de la Política Generalizada (GPI), alternando entre:

- **Evaluación** de política mediante el promedio de retornos.
- **Mejora** de política, haciéndola voraz o ϵ-voraz respecto a la tabla Q.

En la variante **On-Policy** se evalúa la misma política que genera las trayectorias. En el enfoque **Off-Policy**, para corregir la discrepancia entre la política de comportamiento b y la objetivo π, se emplea el muestreo de importancia:

$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$.

Diferencias Temporales

Los métodos TD actualizan las estimaciones tras cada transición, combinando ideas de MC y Programación Dinámica mediante la actualización general:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\delta_t.$$

A diferencia de Monte Carlo, TD utiliza bootstrapping, introduciendo cierto sesgo pero reduciendo significativamente la varianza y permitiendo aprendizaje online.

- **SARSA (On-Policy):** El error temporal es $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$. Aprende el valor de la política exploratoria real.
- **Q-Learning (Off-Policy):** El error temporal es $\delta_t = R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)$. Aprende directamente la política óptima independientemente de la exploración.

Comparativa Monte Carlo vs. Diferencia Temporal Ambos enfoques implementan Iteración de la Política Generalizada dentro de un marco estrictamente tabular. La diferencia reside en el mecanismo de estimación:

- **Monte Carlo**: actualización al final del episodio, sin bootstrapping, estimador insesgado pero de alta varianza.
- **TD**: actualización paso a paso, con bootstrapping, menor varianza y mayor eficiencia muestral.

Mientras Monte Carlo resulta natural en entornos puramente episódicos como Blackjack, los métodos TD muestran ventajas prácticas en entornos con penalizaciones intermedias frecuentes, como CliffWalking, donde la actualización inmediata acelera el aprendizaje.

2.3.2. Aproximación de Funciones (Deep RL)

Cuando el espacio de estados es continuo (como en `MountainCar-v0`) la representación tabular es inviable. Para generalizar la experiencia se aproxima la función de valor mediante pesos parametrizados (una red neuronal): $\hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$.

SARSA Episódico Semi-gradiente (On-Policy)

Extiende el método tabular empleando el descenso del gradiente estocástico.

DQN (Off-Policy)

La adaptación de Q-Learning a redes neuronales se enfrenta a la denominada **"Tríada Mortal"** [2]: la combinación de aproximación de funciones, *bootstrapping* y aprendizaje *off-policy* tiende a hacer diverger los pesos de la red. Para estabilizar el aprendizaje, DQN introduce dos modificaciones:

1. **Experience Replay:** Las transiciones $(S_t, A_t, R_{t+1}, S_{t+1})$ se almacenan en un búfer de memoria $\mathcal{D}$. La red se entrena muestreando mini-batches aleatorios, lo que rompe la alta correlación temporal de las secuencias del MDP e impide el olvido catastrófico.
2. **Target Network:** Se utiliza una segunda red neuronal con pesos $\mathbf{w}^-$ para calcular el objetivo temporal, los cuales se congelan y solo se sincronizan con la red principal $\mathbf{w}$ cada cierto número de pasos.

Esta formulación proporciona a DQN un objetivo fijo y descorrelacionado, permitiendo converger en entornos continuos complejos donde las actualizaciones puramente *online* fracasan.

2.4. Evaluación/Experimentos

Tras presentar los algoritmos, se analiza su comportamiento empírico mediante una batería de experimentos orientados a evaluar la convergencia, la estabilidad, la sensibilidad a hiperparámetros y la capacidad de adaptación.

Los experimentos se dividen en dos bloques: evaluación de métodos tabulares y evaluación de métodos con aproximación de funciones.

2.4.1. Configuración Experimental

Entornos Tabulares

- **Blackjack-v1:** 500.000 episodios, $\gamma = 1{,}0$. Métricas: Tasa de éxito (Win Rate) y longitud media del episodio, con una media móvil de 10000 episodios. Se evalúan MC y TD.
- **CliffWalking:** $\gamma < 1{,}0$. Métrica principal: recompensa acumulada por episodio, contrastando el conservadurismo de SARSA frente a la optimalidad de Q-Learning ante penalizaciones severas (-100).

Entorno de Aproximación de Funciones (Continuo)

- **MountainCar-v0:** Espacio de estados continuo y recompensa dispersa (-1 por paso, límite de 200 pasos).
- **Métricas analizadas:** Longitud del episodio, Error Cuadrático Medio (MSE) en las estimaciones de valor temporal, tasa de éxito en evaluación y varianza del retorno mediante diagramas de cajas (*boxplots*). Todo ello promediado sobre 3 semillas aleatorias independientes para garantizar rigor estadístico.

2.4.2. Resultados en entornos tabulares

Blackjack Monte Carlo On-Policy muestra una convergencia estable bajo esquemas de ϵ-decay adecuados. Empíricamente, hemos comprobado que un decaimiento rápido (`decay_rate = 0.0001`) acelera la estabilización inicial, y un decaimiento lento (`decay_rate = 0.00001`) produce mayor varianza inicial, pero políticas finales más robustas. La variante Off-Policy presenta mayor fluctuación durante el entrenamiento debido a la varianza de los pesos del muestreo de importancia, pero, al aislar el aprendizaje en fase de evaluación (apagando la exploración), demuestra haber aprendido perfectamente, alcanzando un *win rate* del 43,22 %, consistente con el máximo teórico del entorno.

Los métodos TD (SARSA y Q-Learning) convergen de forma similar, confirmando que en entornos episódicos sin penalizaciones intermedias, ambos enfoques (MC y TD) resultan adecuados. También hemos visto (2.1) que al estudiar a corto plazo el aprendizaje de los algoritmos TD frente a Monte Carlo On-Policy —es decir, analizando la curva de aprendizaje en la fase inicial del entrenamiento (los primeros 10.000 episodios)—, los métodos TD son algo más eficientes en términos de velocidad de aprendizaje.

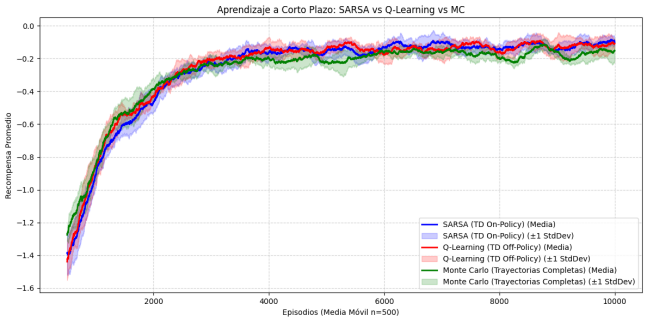

Figura 2.1: Aprendizaje a corto plazo de SARSA, Q-Learning y MC.

Un experimento adicional de aversión al riesgo (modificando la penalización por "bust"de $-1{,}0$ a $-5{,}0$) nos ha mostrado un cambio en la conducta del agente. La longitud media del episodio cae a $f(t) \approx 1{,}05$ pasos y la tasa de victorias en entrenamiento se reduce al $\approx 39\,\%$. Esto evidencia que el agente prioriza plantarse para evitar el castigo más severo, confirmando que la política óptima no es una propiedad intrínseca de las reglas del Blackjack, sino una consecuencia estricta de la función de recompensa que se define. Podemos evidenciar esto en las gráficas 2.2 y 2.3.

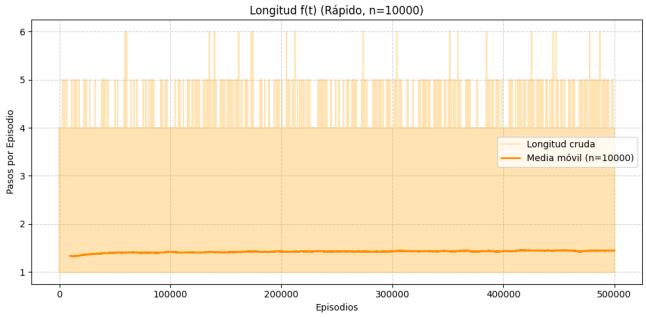

Figura 2.2: Longitud de episodio con media móvil de 10.000 en el agente con decaimiento rápido.

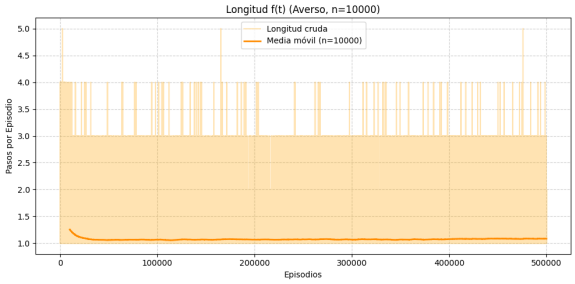

Figura 2.3: Longitud de episodio con media móvil de 10.000 en el agente averso al riesgo.

CliffWalking CliffWalking introduce penalizaciones intermedias severas (-100 al caer al precipicio), lo que permite observar de mejor formas las diferencias entre SARSA y Q-Learning.

SARSA, al ser un método On-Policy, internaliza el riesgo asociado a la exploración ($\epsilon = 0{,}1$). Como consecuencia, converge hacia una política más conservadora que evita transitar cerca del precipicio, incluso si ello implica trayectorias más largas. Esto se traduce en un proceso de aprendizaje con mayor estabilidad y una recompensa promedio superior durante la fase de entrenamiento.

Por su parte **Q-Learning**, al actualizar utilizando $\max_a Q(S_{t+1}, a)$, asume decisiones óptimas futuras y es indiferente ante los errores cometidos por la exploración. Así, aprende la política óptima teórica. Durante el entrenamiento (2.4), esto genera una alta varianza y oscilaciones en la recompensa, ya que el $10\,\%$ de acciones aleatorias provocan inevitablemente múltiples despeñamientos.

Al anular la exploración en la 2.5 para evaluar el rendimiento puro de las políticas extraídas, Q-Learning supera a SARSA obteniendo la máxima recompensa teórica, evidenciando el contraste entre el camino arriesgado pero corto frente al camino seguro pero ineficiente.

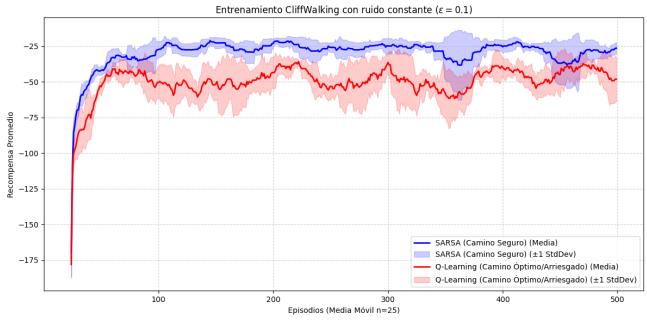

Figura 2.4: Entrenamiento CliffWalking con $\epsilon = 0{,}1$

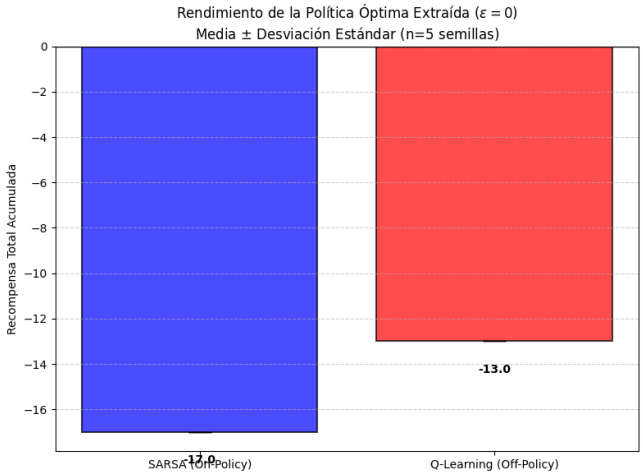

Figura 2.5: Evaluación de la política óptima extraída en CliffWalking con $\epsilon = 0$

Comparativa Global

Desde una perspectiva empírica:

- En Blackjack, MC y TD presentan rendimientos finales similares.
- En CliffWalking, los métodos TD muestran claras ventajas debido a su capacidad de actualización paso a paso.
- Los métodos Off-Policy tienden a converger más directamente hacia la política óptima, aunque con mayor varianza e inestabilidad durante el entrenamiento.
- Los métodos *On-Policy* ofrecen una convergencia más predecible y un comportamiento más seguro durante el aprendizaje, asumiendo una leve pérdida de optimalidad final como margen de seguridad exploratorio.

Estos resultados confirman que la elección entre Monte Carlo y Diferencias Temporales no depende únicamente del marco tabular compartido, sino de la estructura temporal de recompensas del entorno.

2.4.3. Resultados en MountainCar (aproximación de funciones)

Hemos planteado tres estudios para evaluar los límites de las redes neuronales en RL.

Estudio 1: Fracaso de arquitecturas On-Policy puras

Evaluamos SARSA Semi-gradiente probando tres esquemas de exploración: decaimiento lento (tasa = 0,005), rápido (tasa = 0,05) y ϵ constante (0,1).

A pesar de realizar variaciones de los hiperparámetros del agente y probar con más episodios de entrenamiento, el algoritmo no logra resolver el entorno, manteniendo la longitud del episodio estancada en el máximo de 200 pasos en todos los casos. Esto demuestra que el problema no se encuentra en la exploración, sino en la alta correlación temporal de las secuencias del MDP. La actualización *online* paso a paso provoca olvido en los pesos de la red, impidiendo la propagación correcta del valor $Q(s, a)$.

Estudio 2: Comparando SARSA y DQN

Hemos contrastado el rendimiento de SARSA frente a DQN. DQN resuelve el entorno alcanzando una tasa de éxito del 100 % en la fase de evaluación (mediana de 110 pasos, mínimo de 88 pasos), frente al 0 % de SARSA.

También hemos analizado el Error Cuadrático Medio (2.7) durante la fase de entrenamiento, viendo como DQN presenta errores de estimación considerablemente menores que SARSA, lo que sugiere una aproximación más precisa de la función de valor. No obstante, hacia el final del entrenamiento se observa un aumento de la varianza en la pérdida, reflejando la mayor inestabilidad de los métodos off-policy con aproximación mediante redes neuronales.

Estudio 3: Análisis de la "Tríada Mortalz el Replay Buffer

Para demostrar el impacto de romper la dependencia secuencial, hemos realizado un estudio modificando el *Experience Replay* de DQN, limitando su capacidad a 100 transiciones frente al estándar de 10.000.

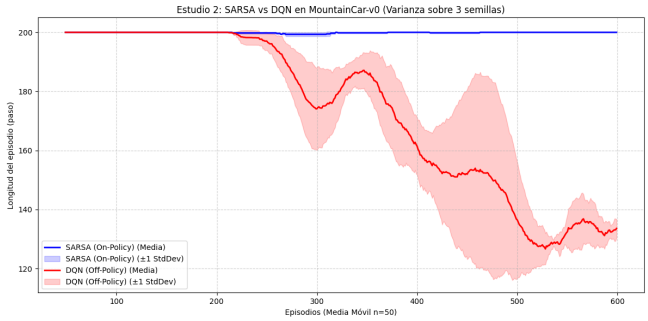

Figura 2.6: Longitud de episodio en SARSA semi gradiente y DQN

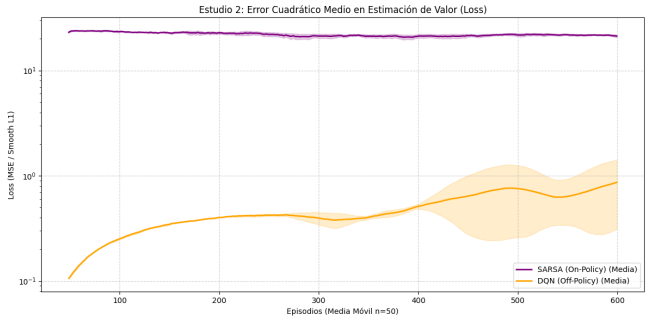

Figura 2.7: Error Cuadrático Medio en Estimación de Valor

Como se observa en el diagrama de cajas de 2.8, el comportamiento de ambas configuraciones es diferente:

- **Buffer estable (10.000):** El agente resuelve el entorno con una tasa de éxito del 100 %. La dispersión de la caja (con episodios que oscilan entre los 85 y casi 170 pasos) indica que, aunque la política es exitosa, aún presenta varianza y requiere más iteraciones para converger a una ruta óptima.
- **Buffer correlacionado (100):** Al forzar a la red a entrenar con mini-batches de transiciones recientes altamente correlacionadas, el agente fracasa en el aprendizaje. El diagrama se mantiene completamente plano en el máximo de 200 pasos, con una tasa de éxito del 0

Tras esto hemos analizado el MSE de ambos enfoques y hemos obtenido un resultado que creemos que es muy interesante y que nos ha costado un poco comprender. En la gráfica 2.9 el agente con buffer bajo converge rápidamente a un error estable y casi nulo. Pero esto no ocurre por optimalidad, sino por pesimismo, ya que la red sobreajusta sus pesos a la dinámica local del fondo del valle, prediciendo con exactitud que siempre recibirá la penalización continua de -1. En cambio, el agente con buffer alto una subida en su error en el momento exacto en el que descubre la meta, evidenciando cómo la red neuronal debe reestructurar sus parámetros preestablecidos para asimilar una recompensa positiva diferente.

2.5. Conclusiones

Tras la implementación y el análisis tanto de métodos tabulares en entornos discretos como de métodos de aproximación de funciones en espacios continuos, se extraen las siguientes conclusiones:

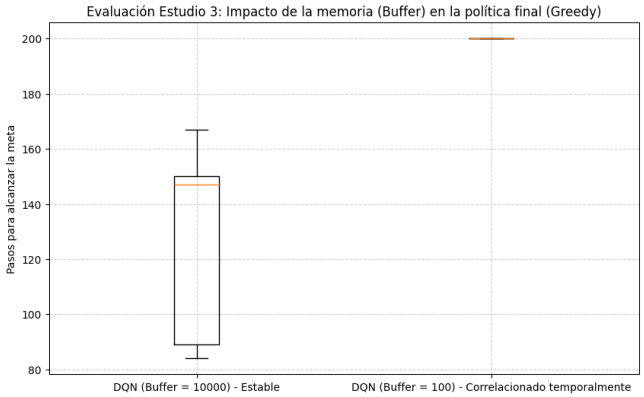

Figura 2.8: Impacto del Buffer en la política aprendida

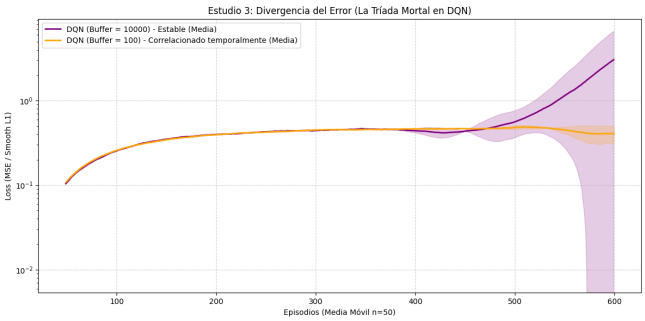

Figura 2.9: Comparación MSE en el estudio 3

- **Eficacia de los métodos tabulares:** Para entornos con espacios de estados discretos y manejables, las técnicas tabulares resultan altamente eficaces. Tanto Monte Carlo como los métodos de Diferencia Temporal lograron converger hacia políticas óptimas o cercanas al óptimo sin necesidad de conocer explícitamente la dinámica del entorno. Esto confirma la validez práctica del marco de Iteración de la Política Generalizada (GPI).
- **Monte Carlo vs. Diferencias Temporales:** En Blackjack, un entorno puramente episódico y sin penalizaciones intermedias frecuentes, Monte Carlo y TD mostraron rendimientos finales similares, aunque MC requirió un número elevado de episodios para estabilizar la tabla Q debido a su alta varianza. En contraste, en CliffWalking —donde existen penalizaciones intermedias severas— los métodos de Diferencia Temporal demostraron ventajas claras al actualizar paso a paso, acelerando el aprendizaje y permitiendo una adaptación más rápida a la estructura de recompensas.
- **On-Policy vs. Off-Policy: estabilidad frente a optimalidad:** Se ha comprobado empíricamente que los enfoques *On-Policy* internalizan el riesgo exploratorio, generando políticas de aprendizaje más estables y conservadoras. Los métodos *Off-Policy* (Q-Learning) apuntan directamente a la optimalidad a costa de asumir una mayor varianza e inestabilidad temporal durante la fase de entrenamiento.
- **La recompensa como determinante del comportamiento:** El experimento del wrapper cambiando la recompensa evidenciia que la política óptima no es una propiedad intrínseca del juego, sino de la función de recompensa definida. Al incrementar la penalización por "bust.^{el} agente adopta una estrategia extremadamente conservadora, reduciendo la longitud media del episodio. Por tanto el agente busca optimizar el retorno esperado G_t, y cualquier rasgo conductual aparece directamente de la estructura de incentivos.
- **Compromiso sesgo-varianza y eficiencia muestral:** Monte Carlo, al no utilizar bootstrapping, constituye un estimador insesgado pero de alta varianza, lo que exige un número elevado de episodios para estabilizar las estimaciones. Los métodos TD introducen cierto sesgo al apoyarse en estimaciones intermedias, pero reducen significativamente la varianza y mejoran la eficiencia muestral. La elección entre ambos enfoques depende, por tanto, de la estructura temporal del entorno y del coste computacional disponible.
- **El límite tabular y la necesidad de generalización:** La transición al entorno *MountainCar* muestra el problema de aumentar la dimensionalidad. Ante espacios de estados continuos, debemos realizar la parametrización del valor mediante redes neuronales, lo que introduce retos de convergencia. El fracaso de SARSA semi-gradiente demuestra que las arquitecturas puramente *online* paso a paso sufren de olvido catastrófico debido a la fuerte correlación temporal de las observaciones espaciales.
- **Estabilización en Deep RL y la "Tríada Mortal":** Combinar la aproximación de funciones, *bootstrapping* y aprendizaje *off-policy* nos lleva a la divergencia matemática. Al resolver el entorno continuo mediante DQN vemos que mecanismos como el *Experience Replay* y la *Target Network* no son solo optimizaciones, sino requisitos algorítmicos estructurales. Reducir el buffer demuestra que, sin decorrelación temporal, la red neuronal colapsa hacia un pesimismo extremo, incapaz de propagar correctamente la señal de recompensa de la meta.

Líneas Futuras

A partir de los resultados obtenidos, se plantean las siguientes direcciones de estudio:

- **Reducción de varianza:** Implementar Weighted Importance Sampling para estabilizar las actualizaciones en el enfoque Off-Policy.
- **Mitigación del sesgo de maximización en Deep RL:** Dado que DQN hereda la sobreestimación del operador máx de Q-Learning, se puede proponer la implementación de arquitecturas *Double DQN* (DDQN) o *Dueling DQN* para refinar la estimación de la función de valor en entornos más estocásticos.
- **Algoritmos de Actor-Crítico para acciones continuas:** En este estudio nos limitamos a la versión discreta de *MountainCar* debido a una restricción matemática de los métodos basados en valor (como DQN o SARSA). Calcular cuál es la mejor acción posible resulta inabarcable cuando el espacio de acciones es continuo y existen infinitas opciones (por ejemplo, pisar un acelerador con distintos grados de fuerza).Para poder resolver la variante continua del entorno (*MountainCarContinuous-v0*), deberíamos cambiar hacia arquitecturas de Actor-Crítico. En estos modelos una red neuronal decide la acción en un rango continuo (el actor) y otra red evalúa la calidad de dicha acción (el crítico).

Bibliografía

[1] RepositorioGitHub. (2026). *CasalsFranco* [Repositorio de software]. GitHub. Enlace en https://github.com/AngelD222/CasalsFranco

[2] Sutton, R. S. y Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2ª ed.). MIT Press.

[3] OpenAI. (2026). *ChatGPT (Versión GPT-5)*. https://chatgpt.com. (Nota: Utilizado para escribir fragmentos de código).

[4] Google. (2026). *Gemini (Versión 3.0)*. https://gemini.google.com. (Nota: Utilizado para la depuración, corrección de errores y optimización de código).

[5] Robbins, H. (1952). *Some aspects of the sequential design of experiments*. Bulletin of the American Mathematical Society, 58(5), 527–535.